# CSS4005 — Cloud Infrastructure Construction

## Lecture 10: Docker and Containers

Nauruzbayeva Farikha · Lesson 10

# Recap — Lesson 9 (Load Balancing and Auto Scaling)

- What does an ALB send traffic to? (a target group — only to targets that pass the health check)
- What does a launch template describe? (what every new instance looks like: AMI, type, security group, user data)
- What do min, desired and max mean in an Auto Scaling group? (never fewer, how many right now, never more)
- An AZ goes down. Who stops the traffic, who replaces the servers? (the ALB stops routing to them; the ASG launches replacements in the other AZ)

# Today’s agenda

- ☐ Why containers: the problem with "install it in user data"
- ☐ Containers vs virtual machines
- ☐ Images, containers, registries and the core commands
- ☐ Writing a Dockerfile, layers and the build cache
- ☐ State, volumes, networks and Docker Compose
- ☐ Containers on AWS: ECR, EC2 + Docker, ECS and Fargate, EKS
- ☐ Security and cost → today’s practice

# The scenario

Last week every instance in the Auto Scaling group ran a user data script: install nginx, install Python, download the app, configure it. Each new server takes minutes to become healthy. One day a package mirror is slow, the next day a library has a new version, and two "identical" servers are not identical any more.

**Today: build the app once into an image, and run exactly that image everywhere — on a laptop, on EC2, on ECS.**

# Block 1

Why containers

# The problem with installing at boot

- **Slow:** every new instance downloads and installs everything again. During a traffic spike, that is exactly when you can’t wait.
- **Drift:** a mirror or a library changes between Monday and Friday, so servers launched on different days run different code.
- **"Works on my machine":** the developer’s laptop, the test server and production all have slightly different versions.
- **Hard to roll back:** "undo" means re-running a different script and hoping it removes what the old one installed.

A container image packs the app **and** everything it needs into one versioned file. Starting it is seconds, and the same image runs the same way everywhere.

# Containers vs virtual machines

| | Virtual machine (EC2) | Container |
|---|---|---|
| What's inside | A whole OS with its own kernel | The app and its libraries only |
| Shares with the host | The hardware (through a hypervisor) | The host's **kernel** |
| Starts in | Tens of seconds to minutes | Usually under a second |
| Typical size | Gigabytes | Tens to hundreds of megabytes |
| Isolation | Strong (separate kernel) | Process-level (namespaces, cgroups) |

Not either-or: in AWS, containers usually run **on** virtual machines — your EC2 instances, or VMs that AWS manages for you (Fargate).

# How isolation works (one slide of Linux)

- **Namespaces** give a process its own view of the system: its own process list, network interfaces, hostname and filesystem.
- **cgroups** limit what it can use: CPU, memory, I/O.
- A container is just a normal Linux process with namespaces and cgroups around it. That's why it starts so fast — there's no OS to boot.
- Docker on macOS or Windows runs a small Linux VM in the background, because containers need a Linux kernel.

# Block 2

Images, containers and registries

# Image vs container

- An **image** is a read-only template: files, settings and the command to run. Like an AMI.
- A **container** is a running (or stopped) instance of an image, with a thin writable layer on top. Like an EC2 instance launched from that AMI.
- One image → many containers. Delete a container, the image stays.
- Images have a name and a **tag**: `python:3.12-slim`, `nginx:1.27`, `narxoz-web:v1`. No tag means `:latest` — which is just a name, not "the newest".

# Registries

- A **registry** stores and shares images. You `push` to it and `pull` from it.
- **Docker Hub:** the default public registry ( `nginx` , `postgres` , `python` all come from there).
- **Amazon ECR** (Elastic Container Registry): private registries inside your AWS account, protected by IAM — what EC2 and ECS pull from.
- The image name tells Docker which registry to use:

```
nginx:1.27                                              → Docker Hub
123456789012.dkr.ecr.us-east-1.amazonaws.com/narxoz-web:v1 → your ECR
localhost:5001/narxoz-web:v1                            → a local registry
```

# The core commands

```
docker pull nginx:1.27                          # download an image
docker run -d --name web -p 8080:80 nginx:1.27   # start a container in the background
docker ps                                       # running containers (-a: all)
docker logs -f web                              # its output (Ctrl+C to stop following)
docker exec -it web sh                          # a shell inside it
docker stop web && docker rm web                # stop, then delete the container
docker images                                   # images on this machine
```

`-p 8080:80` = host port 8080 → container port 80. Without `-p`, nothing outside the container can reach it.

# Quick check

You run `docker rm web`. Is the `nginx:1.27` image gone too?

No. `rm` deletes the container; the image stays and can start new containers. `docker image rm` deletes an image.

# Block 3

Building your own image

# The app

A tiny web app ( `app.py` ): it answers with its hostname and a color from an environment variable — so we can see *which* container answered.

```
import os
import socket

from flask import Flask

app = Flask(__name__)
COLOR = os.environ.get("APP_COLOR", "blue")


@app.get("/")
def hello():
    return f"Hello from {socket.gethostname()} | color={COLOR} | v1\n"


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

`requirements.txt` has one line: `flask==3.1.3`

# The Dockerfile, line by line

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN useradd --create-home appuser
COPY app.py .
ENV APP_COLOR=blue
EXPOSE 8000
USER appuser
CMD ["python", "app.py"]
```

- `FROM` the base image · `WORKDIR` the folder inside · `COPY` files in · `RUN` a command at **build** time
- `ENV` a default setting · `EXPOSE` documents the port (doesn't publish it)
- `USER` don't run as root · `CMD` what runs when the container **starts**

# Build and run

```
docker build -t narxoz-web:v1 .
docker run -d --name blue  -p 8001:8000 narxoz-web:v1
docker run -d --name green -p 8002:8000 -e APP_COLOR=green narxoz-web:v1
curl localhost:8001     # Hello from 4a528be586a0 | color=blue | v1
curl localhost:8002     # Hello from bf65d706f072 | color=green | v1
```

- One image, two containers, two different settings: the **configuration comes from the environment**, not from the image.
- The hostname is the container ID — every container is its own "server".

# Layers and the build cache

- Every instruction in a Dockerfile creates a **layer**. `docker history narxoz-web:v1` shows them and their sizes.
- On rebuild, Docker reuses a layer if its instruction **and everything before it** are unchanged.
- That's why `requirements.txt` is copied and installed **before** the code: change `app.py`, and `pip install` comes from the cache.

```
Step  COPY requirements.txt   CACHED
Step  RUN pip install ...     CACHED     ← seconds, not minutes
Step  COPY app.py             runs again (the code changed)
```

Copy everything first ( `COPY . .` ) and any code change reinstalls every dependency.

# `.dockerignore` and image size

- `.dockerignore` keeps files out of the build: `.git`, `__pycache__`, `.env`, local data. Smaller, faster — and no secrets copied in by accident.
- Choose small base images: `python:3.12-slim` instead of `python:3.12`; `alpine` variants where they work.
- Smaller images pull faster on every new instance — this matters during scale-out.

```
# .dockerignore
__pycache__/
*.pyc
.git
.env
```

# Quick check

You change one line in `app.py` and rebuild. Which steps run again?

`COPY app.py` and every step after it. The base image, `COPY requirements.txt` and `pip install` come from the cache.

# Block 4

State, networks and Compose

# Containers should be stateless

- A container's writable layer disappears when the container is deleted. Treat containers as **cattle**: replaceable at any time.
- So where does data go? Outside the container:
  - files and uploads → **S3** (week 5)
  - the database → **RDS** (week 7)
  - a disk that must survive → a **volume** (on EC2: backed by EBS)
- Exactly the rule from week 9: behind a load balancer, any server can answer any request.

# Volumes vs bind mounts

| | Named volume | Bind mount |
|---|---|---|
| Example | `-v dbdata:/var/lib/postgresql/data` | `-v ./config:/etc/app:ro` |
| Managed by | Docker | You (a host folder) |
| Typical use | Database files, data that must survive | Config files, code during development |
| Survives `docker rm` | Yes | Yes (it's your folder) |

`docker compose down` keeps named volumes. `docker compose down -v` deletes them — and your data with them.

# Networks and ports

- Containers on the same Docker network reach each other **by name**: the app connects to `db:5432`, not to an IP. Docker runs a small DNS for that.
- `-p` / `ports:` is only for traffic **from outside** the host. The database doesn't need a published port at all.
- Same idea as week 4/7 security groups: publish only what users must reach; keep the database internal.

# Docker Compose: the whole app in one file

```yaml
services:
  web:
    build: .
    ports:
      - "8080:8000"
    environment:
      APP_COLOR: orange
    depends_on:
      - db

  db:
    image: postgres:17
    env_file: .env          # POSTGRES_PASSWORD=... (not committed)
    volumes:
      - dbdata:/var/lib/postgresql/data

volumes:
  dbdata:
```

`docker compose up -d` · `docker compose ps` · `docker compose logs -f` · `docker compose down`

# Block 5

Containers on AWS

# Amazon ECR: your private registry

```
aws ecr create-repository --repository-name narxoz-web \
  --image-scanning-configuration scanOnPush=true
ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
REG=$ACCOUNT.dkr.ecr.us-east-1.amazonaws.com
aws ecr get-login-password | docker login --username AWS --password-stdin $REG
docker tag narxoz-web:v1 $REG/narxoz-web:v1
docker push $REG/narxoz-web:v1
aws ecr describe-image-scan-findings --repository-name narxoz-web --image-id imageTag=v1
```

- Access is controlled by **IAM**: no separate registry password to manage.
- **Scan on push** checks the image for known vulnerabilities (CVEs).
- Tag with versions ( `v1` , a git commit hash), not just `latest` .

# Build for the right CPU

- Apple Silicon laptops build **arm64** images by default. Most EC2 types we use (t3, m5) are **x86_64 / amd64**.
- An arm64 image on an amd64 instance fails with `exec format error`.
- Build for the target explicitly:

```
docker buildx build --platform linux/amd64 -t narxoz-web:v1 --load .
docker image inspect narxoz-web:v1 --format '{{.Architecture}}'   # amd64
```

(Or use Graviton instances — t4g, m7g — which are arm64.)

# Option 1: EC2 + Docker (your week 9 fleet)

```
#!/bin/bash
dnf install -y docker
systemctl enable --now docker
REG=123456789012.dkr.ecr.us-east-1.amazonaws.com
aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin $REG
docker run -d --restart unless-stopped -p 80:8000 \
  -e APP_COLOR=green $REG/narxoz-web:v1
```

- This becomes the **user data** in last week's launch template. Every instance runs the same image — no more "install at boot".
- The instance pulls from ECR with its **instance profile** (an IAM role): no access keys on the server.
- New version: push `v2`, update the launch template, **instance refresh**.

# Option 2: Amazon ECS

- **Task definition:** the recipe for a container (or a few) — image, CPU and memory, ports, environment, logs, IAM role.
- **Service:** keeps *N* copies (tasks) of a task definition running, replaces failed ones, and registers them in an **ALB target group**.
- **Cluster:** the place where tasks run.
- Like ASG + launch template, but for containers instead of instances.

# ECS launch types: EC2 vs Fargate

| | ECS on **EC2** | ECS on **Fargate** |
|---|---|---|
| Who runs the servers | You (an ASG of container hosts) | AWS — no servers to see |
| You patch and scale hosts | Yes | No |
| Billing | The EC2 instances, used or not | Per task: vCPU and memory per second |
| Good for | Steady, large workloads; special instance types | Most web apps and APIs; spiky load |

With Fargate, each task gets its own network interface, so the ALB target group uses **target type** `ip`, not `instance`.

# ECS service behind an ALB

```
Internet ─► ALB (public subnets, 2 AZs)
              └─► target group (type: ip, port 8000)
                    ├─► task 1  narxoz-web:v1  (private subnet, AZ-a)
                    └─► task 2  narxoz-web:v1  (private subnet, AZ-b)
ECS service: desired count 2, replaces failed tasks, rolling deploys
```

- Same architecture as week 9 — the ALB and the health checks don't change.
- What changes: the unit is a **task** (container), not an instance, and there's no AMI or user data to maintain.
- New version: register a new task definition revision → the service rolls it out, task by task.

# Option 3: Kubernetes (EKS) — a preview

- **Kubernetes** is the open-source standard for running containers on many machines: pods, deployments, services.
- **Amazon EKS** runs the Kubernetes control plane for you; the containers run on EC2 nodes or Fargate.
- More powerful and portable than ECS, but more to learn and operate.
- **App Runner:** the simplest option — point it at an image, get an HTTPS URL, no cluster at all.

# Which one?

| You need | Choose |
|---|---|
| Learning, one small app, full control | EC2 + Docker |
| A web app or API on AWS, little ops work | **ECS on Fargate** |
| Many services, already know Kubernetes, multi-cloud | EKS |
| "Just run this image with a URL" | App Runner |
| Steady heavy load, cost-tuned hosts | ECS on EC2 |

All of them pull images from a registry (usually ECR) and sit behind a load balancer the way you built it in week 9.

# Quick check

An ECS service on Fargate has desired count 3. One task crashes. What happens, and what changes for the ALB?

The service starts a new task to get back to 3, and registers its IP in the target group. The ALB sends traffic to it once it passes the health check — the same self-healing as an ASG, at container level.

# Block 6

Security and cost

# Container security checklist

- **Don’t run as root:** `USER appuser` in the Dockerfile.
- **Small, current base images:** fewer packages, fewer vulnerabilities. Rebuild regularly to pick up fixes.
- **Scan images:** ECR scan on push; fix critical findings before deploying.
- **Least-privilege roles:** on ECS, the **task role** gives *the app* its AWS permissions; on EC2, the **instance profile**. Never access keys in the image.
- **No secrets in images or in git:** pass them at runtime from **Secrets Manager** or **SSM Parameter Store** (ECS can inject them as environment variables).

# Secrets: wrong vs right

```
# WRONG — anyone who can pull the image can read this
ENV DB_PASSWORD=SuperSecret123
```

Right — in the ECS task definition, injected at start from Secrets Manager:

```
"secrets": [
  { "name": "DB_PASSWORD",
    "valueFrom": "arn:aws:secretsmanager:us-east-1:123456789012:secret:narxoz/db" }
]
```

Image layers are permanent: deleting the secret in a later layer doesn't remove it from the earlier one.

# Cost hygiene

- **ECR:** you pay for stored image data. Add a **lifecycle policy** that expires old untagged images.
- **Fargate:** billed for the vCPU and memory a task requests, per second, while it runs — size tasks to what the app actually needs.
- **ECS on EC2 / EC2 + Docker:** you pay for the instances whether they're busy or not.
- After the lab: stop containers, delete ECR repositories, terminate instances, end the lab.

# Common mistakes

| Symptom | Likely cause |
| --- | --- |
| `exec format error` on EC2 | Image built for arm64, instance is amd64 |
| `no basic auth credentials` on push or pull | Forgot `aws ecr get-login-password \| docker login` |
| Container exits immediately | `CMD` crashed — check `docker logs` |
| `curl localhost:8080` refused | Forgot `-p`, or the app listens on `127.0.0.1` instead of `0.0.0.0` |
| Database empty after restart | Data was in the container, not in a volume — or `down -v` |
| ALB targets unhealthy (Fargate) | Target group type `instance` instead of `ip`, or wrong port |

# Block 7

Today’s practice

# Today’s practice — build once, run anywhere

**Part 1 — everyone, local Docker**

1. Run nginx: ports, logs, exec, rm
2. Write a Dockerfile for the app, build it
3. Two containers, two colors, one image
4. See the build cache work
5. Compose: app + PostgreSQL with a volume that survives `down`

**Part 2**

- **Track A (Learner Lab):** push the image to **ECR**, run it on an EC2 instance that pulls it with its **instance profile** — no keys
- **Track B (no AWS):** a local registry as "ECR", then 3 replicas behind HAProxy, like week 9

Checked orally: show the image, two containers, the cache, the data that survived — and where your image ran.

# By the end of this lesson, you should be able to

- ☐ Explain the difference between a container and a virtual machine
- ☐ Explain image vs container vs registry vs tag
- ☐ Write a Dockerfile and order it so the build cache works
- ☐ Keep containers stateless and put state in volumes, RDS or S3
- ☐ Push an image to ECR and run it on EC2 without access keys
- ☐ Choose between EC2 + Docker, ECS (EC2 / Fargate), EKS and App Runner

# Before next lecture

- ☐ Finish today's practice and delete everything in Learner Lab (ECR repository, instance, security group)
- ☐ Draw last week's NarxozReg diagram again with an **ECS service on Fargate** instead of the Auto Scaling group
- ☐ Notice how many resources you created by hand in weeks 4–10 — next week we write them as code

Next week: describing the whole infrastructure in files, and creating it with one command.

# Next lecture

Infrastructure as Code with Terraform — your VPC, ALB and servers, written as code.